La historia de Alpha Centauri · cómo construimos el proyecto y qué aprendimos por el camino

Equipo Alpha Centauri · Rocket Landing · María Jimena Castaño Albarracín, Isaías Céspedes Novoa y Jorge Horacio Rojas Criollo · octubre y noviembre de 2026. Este documento cuenta el proceso en orden: qué hicimos cada semana, qué salió mal, qué cambiamos y por qué. Las fechas y los números salen del historial de git, del catálogo y de las fichas versionadas. Sirve para contar el proyecto como una historia y para no olvidar, al exponer, que cada decisión tuvo un motivo.

Antes de empezar: las palabras que vamos a usar

- **Misión.** Lo que pide la ficha del curso: aterrizar un cohete en una plataforma con velocidad e inclinación seguras y combustible limitado. Todo lo demás existe para cumplirla y para poder demostrar que se cumple.
- **Episodio (en el código, "época").** Un vuelo completo: empieza a 600 m con viento, temperatura y presión sorteados y termina cuando el cohete toca el suelo, sale de la zona, choca con un obstáculo o agota su tiempo.
- **Paso.** Cada decisión del agente. Un paso dura hasta que el cohete se desplaza 20 m en vertical o pasan 2 s; dentro de él la física avanza cada 0,1 s. Un episodio tiene 50 pasos como máximo. Agotarlos sin tocar el suelo es un fracaso especial: el episodio se **trunca**, no termina.
- **Decisiones del agente.** Cuánto empuje dar y hacia dónde inclinarse. El contrato del ambiente acepta cinco números (motor, acelerador, dirección del empuje); nuestros agentes deciden tres: empuje respecto al peso e inclinación en dos ejes. Esa traducción fue la decisión técnica que más importó.
- **Recompensa.** Lo que el ambiente devuelve tras cada paso: una guía con forma de potencial (acercarse, frenar a la velocidad segura, bajar) menos el combustible gastado, y al final +100 por aterrizar, −50 por posarse fuera de la plataforma y −100 por chocar, salirse, colisionar o quedarse sin pasos.
- **Modelo.** Lo que aprende: dos tablas pequeñas en el Q-learning, una red neuronal de 128 × 128 en PPO. Cada modelo entrenado es una **ejecución** con nombre (`ppo-v2-v0.1-s1`), carpeta, huella SHA-256, versión en MLflow y manifiesto.
- **Versionamiento.** La regla del equipo: cualquier cambio de reglas o constantes del ambiente sube su versión (0.1 → 0.2); todo modelo guarda con qué versión y qué commit se entrenó; los modelos de versiones distintas no se comparan. Se versionan el código, las fichas, los modelos, los datos de entrenamiento, el registro de MLflow y el catálogo de lo que se sirve.
- **Experimento.** Un grupo de ejecuciones que responden la misma pregunta cambiando una variable a la vez. Hubo seis.

- **Nivel.** Una versión del mundo más exigente: misión base, pasarela, pasarela con viento fuerte.

Capítulo 1 · Del LunarLander a nuestro propio mundo (23 al 28 de octubre)

El repositorio nació el 23 de octubre con lo más fácil: el `LunarLander` de Gymnasium. Sirvió para entender la interfaz (`reset`, `step`, observación, acción, `terminated`, `truncated`), pero no para la misión: la ficha pedía un mundo en 3D con viento, temperatura, presión y combustible, y queríamos controlar cada variable para poder explicarla.

El 28 de octubre ya existía `RocketLanding3D-v0`. Las decisiones de modelado se tomaron con una pregunta en la cabeza: ¿cómo lo explicamos? Por eso el viento no es una fuerza mágica sino una velocidad del aire que entra por el arrastre, y por eso depende de la densidad, que depende de la temperatura y la presión sorteadas en cada episodio. Por eso el empuje baja con la presión exterior y el combustible se consume con el impulso específico. Y por eso el paso del agente son 20 m o 2 s: desde 600 m hacen falta 30 pasos en línea recta y quedaban 20 de margen. Nuestra primera salida estaba a 800 m y un controlador hecho a mano necesitaba 45 pasos: no cabía. Bajamos a 600.

La recompensa fue la segunda discusión larga. La ficha advertía del reto clásico: un agente que flota acumulando premio sin aterrizar. Lo resolvimos con un potencial: sumada en el episodio, la recompensa de aproximación solo depende de dónde empieza y dónde acaba, así que flotar aporta cero; el combustible resta; agotar los pasos vale −100. Lo comprobamos con políticas fijas antes de entrenar nada: flotar da −205, subir siempre −261, caída libre −113. Aterrizar vale unos +150. Las bandas no se solapan.

Esa misma semana entrenamos el primer modelo, un Q-learning con dos tablas de 297 valores en total, y aprendió: 78 % de aterrizajes tras 5000 episodios, el primero en el episodio 1785. Era la prueba de que el ambiente se podía aprender y evaluar. Y nos dejó la primera lección de MLOps sin buscarla: la primera ejecución `base-v0.1-s0` se había entrenado con cambios sin confirmar en git, el manifiesto lo detectó y la retiramos y la volvimos a entrenar desde un commit limpio. Desde entonces cada manifiesto lleva la marca `*` si hubo cambios sin commit, y el pipeline avisa.

También el 28 nació la escena 3D con three.js. No era adorno: ver el vuelo era la única forma de entender por qué un modelo fallaba.

Capítulo 2 · La red neuronal que no aprendía (28 de octubre)

El experimento 2 sustituyó las tablas por una red neuronal con PPO. La primera versión, `ppo-v0.1-s0`, decidía directamente los cinco valores del contrato. En 400 000 pasos no aterrizó ni una vez: aprendía a acercarse (la recompensa subía de −115 a −49) pero nunca a frenar. La segunda, `ppo-rel-v0.1-s0`, copió la idea del Q-learning y expresó el empuje respecto al peso: frenaba perfectamente, a 0,3 m/s, y chocaba siempre porque llegaba inclinada 17°; le

habíamos permitido hasta 23° y el límite de aterrizaje es 10°, sin ninguna señal intermedia que la corrigiera. La tercera, `ppo-v2-v0.1-s0`, acotó la inclinación a 9,6° y centró la acción en flotar, para que la política inicial no cayera: 97 %.

Lo que aprendimos cabe en tres frases y conviene decirlas en la defensa: la parametrización de la acción decide si el problema es aprendible; los límites de la acción deben respetar las reglas del aterrizaje; arrancar flotando hace útil la exploración desde el primer episodio. Y una cuarta: los dos fracasos siguen en el registro y en MLflow, con su commit y sus métricas, porque son la evidencia.

Con el experimento 2 llegaron MLflow (parámetros, métricas, dataset y modelo registrado por ejecución) y el panel de comparación.

Capítulo 3 · Que se pueda reproducir (30 de octubre al 3 de noviembre)

Isaías entró por la parte que más se ve y por la que menos: una escena realista con nave de acero, cámaras y sonido, y al mismo tiempo las dependencias fijadas a las versiones del entrenamiento, huellas SHA-256 de cada artefacto en el manifiesto, el comando `cli.py verificar` y las primeras pruebas de contrato del ambiente y de recuperación de modelos guardados.

El 2 de noviembre hicimos el primer diagnóstico frente al enunciado del proyecto final y la rúbrica. Nos faltaba lo que convierte un experimento en un sistema: una API que sirviera los modelos, un catálogo de qué modelo está en producción, una web para el docente y un procedimiento de rollback. En dos días salieron la API FastAPI (catálogo, modelos históricos, evaluación en vivo con el mismo ambiente del entrenamiento) y la web Vite + TypeScript + three.js. El catálogo nació con dos operaciones y una regla: `recomendar` promueve un modelo guardando el anterior, `restaurar` es el rollback, y cada cambio deja una línea en su historial con fecha y motivo.

El despliegue tuvo su propia historia. Primero una imagen Docker y un script para Hugging Face Spaces; los Spaces con Docker exigían suscripción. Pasamos a Render en el plan gratuito, con 512 MB: la API no podía importar MLflow entero, así que lee las versiones de modelo de `mlflow.db` con sqlite3. La web fue a Vercel. El 3 de noviembre cerramos el hito 1 con la etiqueta `v0.2.0` y el mismo día el hito 2: hipótesis, cambio, presupuesto y decisión por configuración, manifiestos con el formato de la sección 11 de la guía y suites de evaluación con nombre. La suite v1 quedó fijada: 200 episodios con semillas desde 10 000 000, meteorología que nunca aparece en el entrenamiento, en dos condiciones: normal y viento fijo de 15 m/s.

Capítulo 4 · Dos tropiezos útiles (4 de noviembre)

El servicio de MLflow en Render no arrancaba: `mlflow server` levantaba diez procesos y 1,6 GB, y además rechazaba el host público. Lo resolvimos arrancando uvicorn directamente sobre la aplicación de MLflow, con un solo proceso de 250 MB. Desde entonces cada ficha de

la web enlaza a su run en MLflow.

El segundo tropiezo fue nuestro. Al llevar la escena 3D a la web la habíamos simplificado, y en la revisión se notó: el cohete había perdido calidad. Se repuso la escena cinematográfica completa y el equipo fijó una regla: la escena nunca se degrada para ganar comodidad. Esa regla la respetaron todos los cambios posteriores.

Capítulo 5 · El nivel 2 y el pipeline (5 de noviembre)

Para el nivel 2 escribimos tres candidatos: rotación real con torque, plataforma desplazada con combustible justo, y obstáculos. El equipo eligió obstáculos porque la decisión se ve en el replay. El primer diseño fueron dos torres bajas a un lado; medimos y el modelo del nivel 1, volando a ciegas, solo chocaba el 2 % de las veces porque ya estaba centrado mucho antes de bajar a esa altura. No había decisión. Pasamos a tres torres altas alrededor de la plataforma (ambiente 0.2).

Los resultados contaron una historia clara. PPO desde cero con la receta del nivel 1: 8 %. Partiendo de los pesos del modelo del nivel 1 (las nueve entradas nuevas empiezan en cero, la red arranca comportándose como la original): 76,5 % con el mismo presupuesto. Con 1,5 millones de pasos: 86 %. Esquivar exige anticipar la ruta y 600 000 pasos no bastaban. La transferencia pasó a ser la herramienta central del proyecto.

Esa misma noche cerramos el pipeline con Prefect: ocho etapas que ejecutan los mismos comandos que usábamos a mano (validar el ambiente, probar el contrato, entrenar, evaluar en la suite, registrar, decidir, publicar el catálogo con commit y push, comprobar que la API pública sirve lo esperado), con un criterio de aceptación escrito antes de entrenar: un candidato sustituye al recomendado de su nivel si gana al menos 2 puntos, no pierde más de 5 con viento y no tiene más colisiones. La primera ejecución real promovió `ppo-v2-v0.1-s1` (99 %) sobre la semilla 0 (97 %); la semilla 2 (97,5 %) fue rechazada por el mismo criterio. Prefect no convivía con la versión de FastAPI de la API, así que el orquestador tiene su propio entorno. Cada ejecución deja un JSON de evidencia con las etapas, la decisión y el commit publicado.

Capítulo 6 · El incidente, la intervención y el rollback (5 de noviembre, tarde y noche)

Jorge tomó el nivel 3 con una idea sencilla y potente: el nivel 3 no añade una decisión nueva, cambia las condiciones de operación. Viento de hasta 15 m/s en vez de 10 y ráfagas del doble. Como la observación no cambia, un modelo del nivel 2 juega en el nivel 3 tal cual, y eso permite medir el **incidente**: el recomendado de N2 caía 11 puntos sin reentrenar. La **intervención** fue reentrenar en N3 partiendo de sus pesos: la primera, con 600 000 pasos, fue rechazada por tener más colisiones; con 1,5 millones, 86,5 %; partiendo del nuevo recomendado de N2, 92 %. Tres semillas: 89,3 ± 4,2 %.

Y el **rollback** se demostró de verdad en producción: catálogo v8 (recomendado de N3) → v9 (restaurar el anterior, commit, push, Render redespliega, `comprobar_api.py` deja evidencia de que `GET /version` sirve el catálogo nuevo) → v10 (volver al recomendado). Tres minutos por ciclo. La vista `#/incidente` de la web mostró desde entonces la degradación y la intervención.

Ese día también aprendimos que el pipeline no puede correr en dos máquinas a la vez: `mlflow.db` vive en git, es binario y no se fusiona. Hubo un push rechazado, se fusionó a mano el registro y se volvió a registrar la ejecución en MLflow.

Capítulo 7 · Cuando el profesor pidió ver la complejidad (6 de noviembre)

El 6 de noviembre llegaron dos peticiones: que en la web se pudiera ver cada experimento, cambiar entre ellos para comparar cómo crece la complejidad, ver sus variables, y que la pantalla fuera adaptable; y un archivo con dónde se guarda cada modelo y qué cambia en cada experimento. Salieron `EXPERIMENTOS.md`, generado por un script a partir de los archivos versionados para que nunca mienta, y la vista Experimentos. La portada siguió siendo el cohete, ahora con pestañas de experimento y fichas de modelo.

El mismo día el equipo tomó la decisión más visible del proyecto: cambiar las torres por una **pasarela** con una sola abertura desplazada. El motivo era de diseño experimental, no estético: con las torres la ruta recta solo cruzaba una torre en el 30 % de las salidas y el modelo de N1 a ciegas conservaba el 72 %; con una pasarela que cubre todo el cuadrado de salida, la ruta recta choca siempre salvo que la salida caiga sobre la abertura, y el modelo de N1 a ciegas baja al 5 %. La decisión es inevitable: entrar por la abertura y trasladarse 30 m por debajo mientras se frena desde 180 m. Pedimos que la pasarela no tapara al cohete: se dibuja translúcida y se vuelve casi transparente cuando queda entre la cámara y el cohete. Las torres (0.2 y 0.3) quedaron retiradas pero reproducibles: sus modelos siguen en el catálogo como históricos.

El primer entrenamiento bajo la pasarela no aprendía: 2 % a los 320 000 pasos. La observación daba la posición de la abertura y la red heredada seguía volando hacia la plataforma, que es lo que sabía hacer. Lo abortamos, borramos la run de MLflow y cambiamos dos cosas: la observación pasó a dar el **objetivo de la fase** (la abertura mientras estás encima, la plataforma una vez debajo, como un sistema de navegación que da el siguiente punto de paso), y la transferencia pasó a heredar en esas entradas los pesos de la posición, con signo cambiado. La política de partida ya aterrizaba el 35 % sin entrenar. Lección: cuando el obstáculo cambia qué significa "objetivo", hay que dárselo al agente en la observación.

Isaías convirtió el replay en una consola de misión con los criterios de aterrizaje en vivo, sonido sintetizado del motor y del viento, y una landing de presentación.

Capítulo 8 · Tres semillas, cuatro intervenciones y un modelo final (8 de noviembre)

Jorge pasó la pasarela por el pipeline desde su máquina, lo que obligó a que MLflow se pudiera registrar desde cualquier equipo. Las tres semillas del nivel 2 v1 dieron 91,5, 97,5 y 100 %: media 96,3 ± 4,4. El pipeline las fue promoviendo una tras otra porque cada una mejoraba en más de 2 puntos a la anterior; el recomendado quedó en la semilla 2, con cero colisiones.

El incidente del nivel 3 v1 se midió para las tres: caían 27, 12,5 y 6,5 puntos. La intervención desde la semilla 0 dio 94,5 % y fue promovida; sus semillas 1 y 2 (94 y 92,5 %) fueron rechazadas por el criterio. Y entonces pasó algo que solo un pipeline con criterio fijo pone en evidencia: el nuevo recomendado de N2 (semilla 2) jugando en N3 sin reentrenar daba 93,5 %, a un punto del recomendado de N3. La intervención ya no se justificaba desde ahí. Repetimos la receta cambiando solo el modelo de partida: 97,5 % de aterrizajes, 3 colisiones y 85,5 % con viento de 15 m/s, 24 puntos más que el mismo modelo sin reentrenar. Ese es el recomendado de N3 y el catálogo quedó en su versión 15.

Dos cosas más aprendimos ese día. Partir del mejor modelo del nivel anterior importa más que el presupuesto. Y reentrenar desde un modelo que ya sabe usar la abertura reduce la variabilidad entre semillas: ±1,0 frente a ±4,4.

Capítulo 9 · Cerrar como un sistema, no como un experimento (10 y 11 de noviembre)

El 10 de noviembre se añadió el monitoreo (`GET /monitoreo` y su vista: qué commit y catálogo se sirven, qué modelo atiende cada nivel y cómo se comporta, las últimas decisiones del pipeline y los rollbacks, la latencia de las evaluaciones servidas y alertas), se escribió el informe final y el guion de la defensa, y se aplicó **una sola vez** el test reservado: una suite con la misma estructura que la v1 pero con una semilla que no está en el repositorio. Los tres recomendados dieron 97,5, 99,0 y 97,0 %, entre 0,5 y 1,5 puntos por debajo de la suite v1: la selección no sobreajustó. Etiqueta `v1.0.0`.

El 11, Isaías rediseñó la web como un simulador de aterrizaje en un planeta ficticio, Elyndra, y ese mismo día añadimos lo que faltaba para que el viento se pudiera **ver**: una misión en vivo en la que la API simula paso a paso con el modelo congelado y quien mira mueve el viento con una barra mientras el cohete vuela y sigue reaccionando. No es un experimento ni un entrenamiento: es la perturbación del nivel 3 aplicada a mano. Luego la pantalla completa, el paquete de entregables con el índice rúbrica → evidencia y los PDF generados desde los Markdown, y el 13 los nombres completos, la marca Alpha Centauri en toda la web y el viento en vivo como un modo más del simulador.

Lo que cambiamos por el camino, en una tabla

Qué teníamos	Qué cambiamos	Por qué	Qué aprendimos
LunarLander	mundo propio en 3D	la misión pedía viento, atmósfera, combustible; queríamos explicar cada variable	modelar con la explicación en mente
Salida a 800 m	600 m	con 50 pasos no cabía el frenado	el límite de pasos es una decisión de diseño, no un detalle
Red con acción cruda	acción relativa v2	0 % → 97 % sin tocar el ambiente	la parametrización decide la aprendibilidad
Inclinación libre hasta 23°	≤ 9,6°	chocaba por inclinación	los límites de la acción respetan las reglas
Ejecución con cambios sin commit	retirada y reentrenada	trazabilidad	el manifiesto detecta lo que la memoria olvida
Hugging Face Spaces	Render + Vercel	Docker exigía suscripción; 512 MB	leer `mlflow.db` con sqlite3 en la API
`mlflow server`	uvicorn directo	10 procesos, 1,6 GB, host rechazado	un proceso, 250 MB
Escena simplificada en la web	escena cinematográfica completa	se notó la pérdida	la calidad visual es parte del producto
Dos torres bajas	tres torres altas	2 % de colisiones a ciegas: no había decisión	medir la dificultad antes de entrenar
Entrenar desde cero en N2	transferencia desde N1	8 % → 76,5 % → 86 % con más presupuesto	la transferencia hace abordables los niveles
Decidir a mano qué modelo sirve	pipeline con criterio escrito antes	el criterio no depende de quién mira	las semillas rechazadas también cuentan
Torres	pasarela con abertura desplazada	la decisión debía ser inevitable (72 % → 5 % a ciegas)	diseñar el obstáculo con datos
Posición de la abertura en la observación	objetivo de la fase + herencia de pesos	2 % a los 320 000 pasos	dar al agente lo que significa "objetivo"
Una semilla por configuración	tres semillas, media y desviación	hasta 4 puntos de variabilidad	la cifra honesta es la media
Suite v1 como única medida	test reservado aplicado una vez	descartar el sobreajuste a la suite	−0,5 a −1,5 puntos: no lo hubo
Viento solo en tablas	viento en vivo en la web	que el incidente se pueda ver	el mismo modelo congelado, otra forma de servirlo

Capítulo 10 · Para qué sirve esto fuera del aula

El cohete es un pretexto. Lo que construimos alrededor es lo que una empresa necesita cuando pone un modelo a tomar decisiones y el mundo le cambia debajo de los pies. Cada pieza tiene su traducción:

En el proyecto	En un negocio
El ambiente versionado con contrato y ficha (0.1.1, 0.4, 0.5)	El entorno de datos o el proceso que el modelo atiende: cuando cambian sus reglas o sus rangos, se versiona y se vuelve a validar. Un modelo entrenado con un contrato no se compara con otro.
El modelo como ejecución con nombre, huella SHA-256, commit, configuración y versión en MLflow	El registro de modelos: en una auditoría se puede decir exactamente qué pesos, qué datos y qué código tomaron una decisión el día X.
La suite v1 fija y el test reservado de una sola vez	El conjunto de validación congelado y el holdout que nadie toca: la diferencia entre "funciona en mi evaluación" y "funciona".
El incidente: el viento sube de 10 a 15 m/s y el modelo cae 11 o 27 puntos	La deriva de datos o de concepto: una temporada nueva en la demanda, un patrón de fraude nuevo, un proveedor que cambia una máquina. Primero se mide, luego se actúa.
La intervención con criterio de aceptación escrito antes (+2 puntos, viento −5 como mucho, no más colisiones)	La política de promoción a producción: un candidato no sustituye al modelo vigente porque alguien lo prefiera, sino porque supera umbrales acordados; los rechazos quedan registrados.
El catálogo con recomendado y anterior, y el rollback demostrado en producción (v8 → v9 → v10)	El plan de reversión: si el modelo nuevo falla un lunes a las 9, se vuelve al anterior en minutos con un comando, un commit y una comprobación, no con una reunión.
El pipeline de ocho etapas con evidencia JSON por ejecución	La integración y despliegue continuos de modelos: lo que se hacía a mano se ejecuta igual cada vez y deja rastro.
El monitoreo: qué commit se sirve, cómo se comporta cada modelo, alertas	La observabilidad del servicio: saber qué hay en producción y cuándo empieza a comportarse distinto, antes de que lo diga el cliente.
Tres semillas, media y desviación	La honestidad sobre la incertidumbre: el mejor resultado no es el esperado; decidir con medias evita sorpresas al replicar.
El viento en vivo	La prueba de estrés frente a un público: enseñar cómo reacciona el sistema a condiciones fuera de lo entrenado, en lugar de prometer que no pasará.

Dónde aplica directamente la técnica, además del método: control de procesos industriales y edificios (climatización, hornos, redes eléctricas), donde un agente decide acciones continuas bajo perturbaciones; logística y drones de reparto, donde la ruta cambia con el viento y los obstáculos; y, en general, cualquier sistema donde el costo de una decisión mala

es alto y las condiciones de operación no son las del entrenamiento. En todos ellos el valor no está solo en el modelo que aterriza el 97,5 % de las veces, sino en poder demostrar con qué se entrenó, cómo se evaluó, cuándo dejó de servir, qué se hizo y cómo se volvió atrás.

Lo que haríamos después

Una abertura aleatoria por episodio, para que el modelo no pueda memorizar su posición. Dinámica de rotación real con torque, como propone la ficha. Hacer observable el bloqueo térmico del motor y calibrarlo para que importe. Un servidor de MLflow compartido en vez de la base en git, para que dos personas puedan ejecutar el pipeline a la vez. Y una penalización de "fuera de plataforma" proporcional a la distancia, que es la única trampa de la recompensa que dejamos abierta.